

编译原理-语义分析 01

语义分析基础、语法制导翻译

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 4 月 24 日



福州大学
FUZHOU UNIVERSITY 1958



① 知识点回顾

② 语法制导翻译

③ 语法制导定义

④ 语法制导翻译 (Syntax-Directed Translation, SDT)



① 知识点回顾

语义分析概述
符号表管理

② 语法制导翻译

③ 语法制导定义

④ 语法制导翻译 (Syntax-Directed Translation, SDT)



① 知识点回顾

语义分析概述

符号表管理

② 语法制导翻译

③ 语法制导定义

④ 语法制导翻译 (Syntax-Directed Translation, SDT)

语义分析的三目标

语义分析是编译过程中**验证程序逻辑正确性**的阶段，确保程序符合**语言规范**（如**类型匹配、作用域规则**）。

- ① 静态语义检查；
 - **目的**：在编译阶段识别不符合语言规范的逻辑错误，避免程序运行时崩溃或不可预测行为。
 - **意义**：弥补语法分析的局限性，通过语言语义规则¹ 验证程序结构的逻辑一致性，显著提升代码可靠性。它可在编译时拦截数据类型不匹配等错误，减少调试成本。
 - 例：a[3.14] 语法合法，但语义错误。
- ② 构建**符号表 (Symbol Table)**；
 - **目的**：建立程序实体的结构化信息库，为跨模块的符号引用提供上下文支持。
 - **意义**：符号表是编译器管理标识符的核心数据结构，通过作用域嵌套和类型绑定，确保变量、函数等实体的唯一性和可见性。
- ③ 为中间代码生成或优化提供语义信息；
 - 例如，强类型语言需根据变量类型生成不同的算术指令（如整数加法与浮点加法指令不同），而作用域信息则决定变量的存储周期（如栈帧分配）。

¹类型系统、作用域规则等

语义分析的核心任务

- ① 类型检查：验证表达式、赋值、函数调用等操作的合法性（如整型与浮点型混合运算的隐式转换）。
- ② 作用域分析：解析变量 / 函数的可见性（如局部变量覆盖全局变量）。
- ③ 一致性检查：函数参数数量与声明匹配、结构体成员访问合法性等。
- ④ 控制流验证：检测不可达代码、循环 / 条件语句的合理结构。
- ⑤ 符号表管理：动态维护标识符的语义信息（如类型、内存地址）。



① 知识点回顾

语义分析概述
符号表管理

② 语法制导翻译

③ 语法制导定义

④ 语法制导翻译 (Syntax-Directed Translation, SDT)

核心结构与功能

符号表管理是编译过程中维护程序语义正确性的核心机制。其核心结构为**键值对 (Key-Value)**。以标识符名称（变量、函数名、类型名、类名等）为键，关联类型、作用域、内存地址等属性。

符号表的功能（即操作接口）：

- ① 插入 (声明时注册符号);
- ② 查询 (引用时验证存在性);
- ③ 删除 (退出作用域时清理)。

特别地，这些操作都需要高效进行²。

²需要使用哈希表、二叉查找树、可持久化数据结构等

符号表的实现

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map（TreeMap 和 HashMap）、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；
 - 写时复制（Copy-on-Write, COW）；
 - 可以零成本恢复历史版本；
 - 用空间换时间；
 - 现实中的方案：
 - 顶层浅拷贝：**注意和深度拷贝的区别**；
 - 记录新增和变化量，离开时还原；

类型检查

意义 确保程序中的操作符、表达式及赋值等行为符合语言**类型系统**的约束，避免因**类型不匹配**导致的逻辑错误或运行时崩溃。

方法 通过静态语义分析遍历抽象语法树，验证变量类型是否**兼容**（如整型与浮点型运算是否允许隐式转换），并检查函数参数类型是否与声明一致。若发现非法操作（如字符串与数值相加），编译器将抛出错误。

例子 在 C 语言中，若定义 `int a = 5; float b = 3.14;`，表达式 `a + b` 会被隐式转换为浮点型运算；但若尝试将字符串直接赋值给整型变量，类型检查会阻止此操作。

核心原因 **CPU 寄存器、内存存储时不区分数据类型。由使用数据的 ALU 负责解释数据。**

作用域分析

- 意义** 确定变量、函数等标识符的**可见范围**，防止因作用域嵌套或覆盖引发的语义歧义（如局部变量屏蔽全局同名变量）。
- 方法** 基于符号表构建作用域层级结构（如树形结构），逐层解析标识符的声明位置。编译器在进入新作用域（如函数或代码块）时**创建子符号表**，退出时**销毁**，确保作用域链的正确性。
- 例子** 在 Python 中，若全局变量 $x = 10$ 与函数内局部变量 $x = 20$ 同名，函数内部访问 x 时优先引用局部变量，作用域分析确保这一逻辑的准确性。

一致性检查

- 意义** 确保程序元素的使用与其声明严格一致，避免因参数不匹配或非法访问导致的逻辑错误。
- 方法** 验证函数调用时的参数数量与声明是否一致，检查结构体或对象成员的访问合法性（如是否存在该成员、访问权限是否合规）。
- 例子** 在 C++ 中，若声明函数 `void func(int a, float b)`，调用时若使用 `func(10)` 或 `func("str", 3.14)`，一致性检查会分别报错“参数数量不足”和“类型不匹配”。

控制流验证 (可选)

- 意义** 检测代码中不可达的分支（如永远无法执行的代码块）或非结构化控制流（如循环缺少终止条件），确保程序逻辑的合理性。
- 方法** 构建控制流图（CFG），分析代码块的执行路径可达性。例如，若条件语句的条件恒为真，则其 `else` 分支会被标记为不可达；若函数存在多个 `return` 语句，需验证后续代码是否可能被执行。
- 例子** 在 Java 中，若编写 `while (true) { ... }` 且后续无 `break` 语句，控制流验证会提示循环体后的代码不可达。



① 知识点回顾

② 语法制导翻译 基本概念

③ 语法制导定义

④ 语法制导翻译 (Syntax-Directed Translation, SDT)



① 知识点回顾

② 语法制导翻译 基本概念

③ 语法制导定义

④ 语法制导翻译 (Syntax-Directed Translation, SDT)

语法制导翻译 (Syntax-Directed Translation)

定义 语法制导翻译使用上下文无关文法 CFG (context free grammar) 来引导对语言的翻译，是一种面向文法的翻译技术。

语法制导翻译 (Syntax-Directed Translation)

定义 语法制导翻译使用上下文无关文法 CFG (context free grammar) 来引导对语言的翻译，是一种面向文法的翻译技术。

所处阶段 语法制导翻译横跨了**语法分析、语义分析、中间代码生成**三个阶段。

语法制导翻译 (Syntax-Directed Translation)

定义 语法制导翻译使用上下文无关文法 CFG (context free grammar) 来引导对语言的翻译，是一种面向文法的翻译技术。

所处阶段 语法制导翻译横跨了**语法分析**、**语义分析**、**中间代码生成**三个阶段。

基本思想 当归约 (或推导) 到某个产生式时，除了按照产生式进行相应的代换之外 (语法分析)，还要按照产生式所对应的语义规则执行相应的语义动作，如计算表达式的值、产生中间代码 (语义分析)。

基本思想

① 如何表示语义信息？



基本思想

① 如何表示语义信息？

- 为 CFG 中的**文法符号**设置**语义属性** (如类型 T、值 val 等), 用来表示语法成分对应的**语义信息**。



基本思想

① 如何表示语义信息？

- 为 CFG 中的**文法符号**设置**语义属性** (如类型 T、值 val 等), 用来表示语法成分对应的**语义信息**。

② 如何计算语义属性？



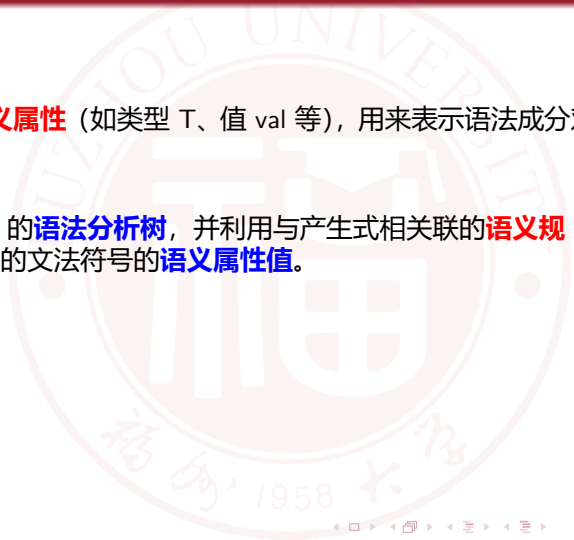
基本思想

① 如何表示语义信息？

- 为 CFG 中的**文法符号**设置**语义属性** (如类型 T、值 val 等)，用来表示语法成分对应的**语义信息**。

② 如何计算语义属性？

- 对于给定的输入串 x ，构建 x 的**语法分析树**，并利用与产生式相关联的**语义规则**来计算分析树中各结点对应的文法符号的**语义属性值**。



基本思想

- ① 如何表示语义信息？
 - 为 CFG 中的**文法符号**设置**语义属性** (如类型 T、值 val 等), 用来表示语法成分对应的**语义信息**。
- ② 如何计算语义属性？
 - 对于给定的输入串 x , 构建 x 的**语法分析树**, 并利用与产生式相关联的**语义规则**来计算分析树中各结点对应的文法符号的**语义属性值**。
- ③ 对输入串的**翻译**就是根据**语义规则**进行**属性计算**。

基本思想

- ① 如何表示语义信息？
 - 为 CFG 中的**文法符号**设置**语义属性** (如类型 T、值 val 等), 用来表示语法成分对应的**语义信息**。
- ② 如何计算语义属性？
 - 对于给定的输入串 x , 构建 x 的**语法分析树**, 并利用与产生式相关联的**语义规则**来计算分析树中各结点对应的文法符号的**语义属性值**。
- ③ 对输入串的**翻译**就是根据**语义规则**进行**属性计算**。
- ④ **语义规则**也叫**语义子程序或语义动作**。

基本思想

- ① 如何表示语义信息？
 - 为 CFG 中的**文法符号**设置**语义属性** (如类型 T、值 val 等), 用来表示语法成分对应的**语义信息**。
- ② 如何计算语义属性？
 - 对于给定的输入串 x , 构建 x 的**语法分析树**, 并利用与产生式相关联的**语义规则**来计算分析树中各结点对应的文法符号的**语义属性值**。
- ③ 对输入串的**翻译**就是根据**语义规则**进行**属性计算**。
- ④ **语义规则**也叫**语义子程序或语义动作**。
- ⑤ 将语义规则同语法规则 (产生式) 联系起来有两种方式：
 - 语法制导定义 (Syntax-Directed Definitions, SDD)
 - 语法制导翻译方案 (Syntax-Directed Translation Scheme, SDT)



① 知识点回顾

② 语法制导翻译

③ 语法制导定义

④ 语法制导翻译 (Syntax-Directed Translation, SDT)

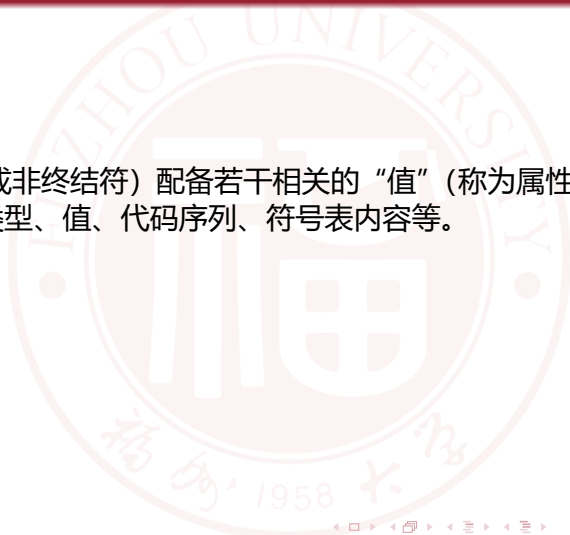
语法制导定义 (Syntax-Directed Definitions)

- SDD 是对 CFG 的推广



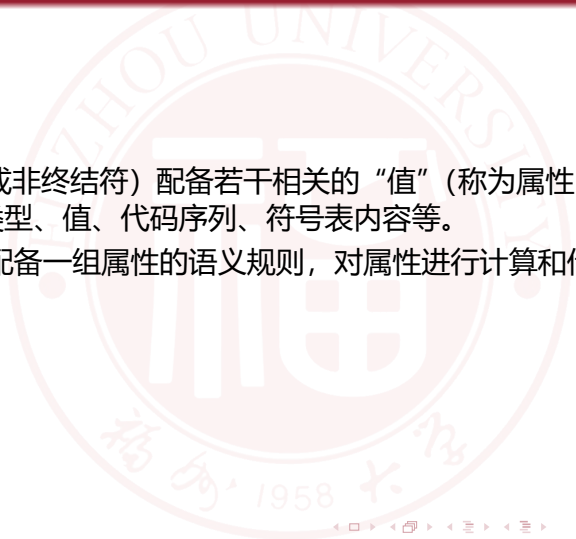
语法制导定义 (Syntax-Directed Definitions)

- SDD 是对 CFG 的推广
- SDD 为每个文法符号 (终结符或非终结符) 配备若干相关的“值” (称为属性), 代表与文法符号相关信息, 如类型、值、代码序列、符号表内容等。



语法制导定义 (Syntax-Directed Definitions)

- SDD 是对 CFG 的推广
- SDD 为每个文法符号（终结符或非终结符）配备若干相关的“值”（称为属性），代表与文法符号相关信息，如类型、值、代码序列、符号表内容等。
- SDD 为每个文法的每个产生式配备一组属性的语义规则，对属性进行计算和传递。



语法制导定义 (Syntax-Directed Definitions)

- SDD 是对 CFG 的推广
- SDD 为每个文法符号（终结符或非终结符）配备若干相关的“值”（称为属性），代表与文法符号相关信息，如类型、值、代码序列、符号表内容等。
- SDD 为每个文法的每个产生式配备一组属性的语义规则，对属性进行计算和传递。
- 如果 X 是一个**文法符号**， a 是 X 的一个**属性**，则用 $X.a$ 表示**属性** a 在某个标号为 X 的**分析树结点**上的**值**。

例 (SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

- **文法符号**: L 、 E 、 T 、 F 、 $digit$
- **属性**: val
- **语义规则**: 通常写成表达式的形式, 用于计算文法符号的属性值

例 (SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

- **文法符号**: L 、 E 、 T 、 F 、 $digit$
- **属性**: val
- **语义规则**: 通常写成表达式的形式, 用于计算文法符号的属性值

文法符号的属性分类:

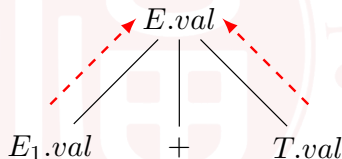
- 综合属性 (Synthesized Attribute)
- 继承属性 (Inherited Attribute)

综合属性

- 在分析树结点 N 上的非终结符 A 的综合属性只能通过 N 的**子结点**或 **N 本身**的属性值来定义。

例

产生式	语义规则
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$



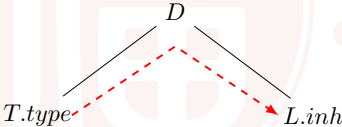
- 终结符可以具有综合属性。**终结符的综合属性值是由词法分析器提供的词法值，在 SDD 中没有计算终结符属性值的语义规则。
 - token: < 种类值, 属性值 (词法值) >

继承属性

- 在分析树结点 N 上的非终结符 A 的**继承属性**只能通过**N 的父结点、N 的兄弟结点或 N 本身**的属性值来定义。

例

产生式	语义规则
$D \rightarrow T L$	$L.inh = T.type$



- 终结符没有继承属性**。终结符从**词法分析器**处获得的属性值被归为综合属性值。

两个属性的综合对比

特征	综合属性	继承属性
信息传递方向	自下而上	自上而下
语法规则	根据右部候选式中的符号的属性 计算左部被定义符号的综合属性	根据右部候选式中的符号的属性和 左部被定义符号的属性 计算右部候选式中的符号的继承属性
对应语法树	根据子结点的属性和父结点自身的属性 计算父节点的综合属性	根据父结点和兄弟节点的属性 计算子结点的继承属性

SDD 综合示例

例 (分析 $3 * 5 + 4n$ 的过程)

产生式	语义规则
$L \rightarrow E n$	<i>print</i> (<i>E.val</i>) ¹
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$ ²

SDD 综合示例

例 (分析 $3 * 5 + 4n$ 的过程)

产生式	语义规则
$L \rightarrow E\ n$	<i>print</i> (<i>E.val</i>) ¹
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$ ²

¹虚拟综合属性、副作用

SDD 综合示例

例 (分析 $3 * 5 + 4n$ 的过程)

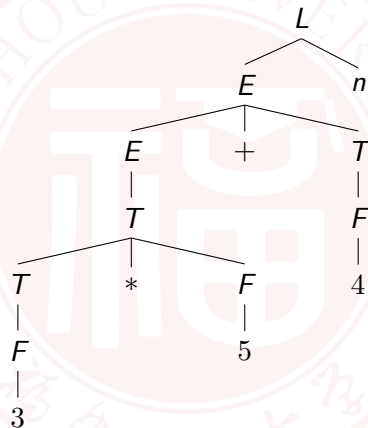
产生式	语义规则
$L \rightarrow E n$	<i>print</i> (<i>E.val</i>) ¹
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$ ²

¹虚拟综合属性、**副作用**

²终结符只有综合属性，由词法分析器提供

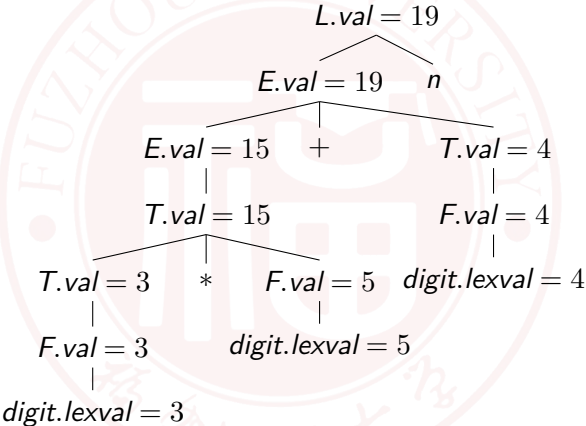
分析 $3 * 5 + 4n$ 的过程

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$



分析 $3 * 5 + 4n$ 的过程

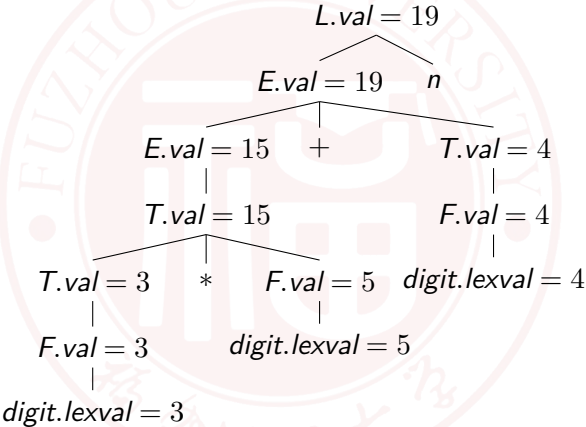
产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$



分析 $3 * 5 + 4n$ 的过程

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

注释分析树 (Annotated Parse Tree)：每个节点都带有属性值的分析树。



SDD 属性的计算

- 对出现在产生式**右边**的**继承属性**和出现在产生式**左边**的**综合属性**都必须提供一个**计算规则**。属性计算规则只能使用本产生式中的文法符号的属性。

例

对于产生式 $F \rightarrow digit$, 左侧文法符号 F 的综合属性 val 由 $F.val = digit.lexval$ 得到。

- 出现在产生式**右边**的**综合属性**和出现在产生式**左边**的**继承属性**不由该产生式所给的属性计算规则进行计算, 由**其他产生式**的属性计算规则或者由属性计算器的参数提供。
- 语义规则所描述的工作可以包括**属性计算**、**静态语义检查**、**符号表操作**、**代码生成**等。

计算规则示例

例

考虑非终结符 A 、 B 和 C ， A 有继承属性 a 和综合属性 b ， B 有综合属性 c ， C 有继承属性 d 。

产生式 $A \rightarrow BC$ 不可能有以下规则的哪一个？

- ① $C.d = B.c + 1$
- ② $A.b = A.a + B.c$
- ③ $B.c = A.a$

计算规则示例

例

考虑非终结符 A 、 B 和 C ， A 有继承属性 a 和综合属性 b ， B 有综合属性 c ， C 有继承属性 d 。

产生式 $A \rightarrow BC$ 不可能有以下规则的哪一个？

- ① $C.d = B.c + 1$
- ② $A.b = A.a + B.c$
- ③ $B.c = A.a$

计算规则示例

例

考虑非终结符 A 、 B 和 C ， A 有继承属性 a 和综合属性 b ， B 有综合属性 c ， C 有继承属性 d 。

产生式 $A \rightarrow BC$ 不可能有以下规则的哪一个？

- ① $C.d = B.c + 1$
- ② $A.b = A.a + B.c$
- ③ $B.c = A.a$

属性 $A.a$ 和 $B.c$ 应用其它产生式的语义规则计算。

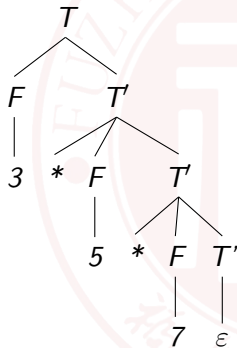
综合示例

例 ($3 * 5 * 7$ 的分析)

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

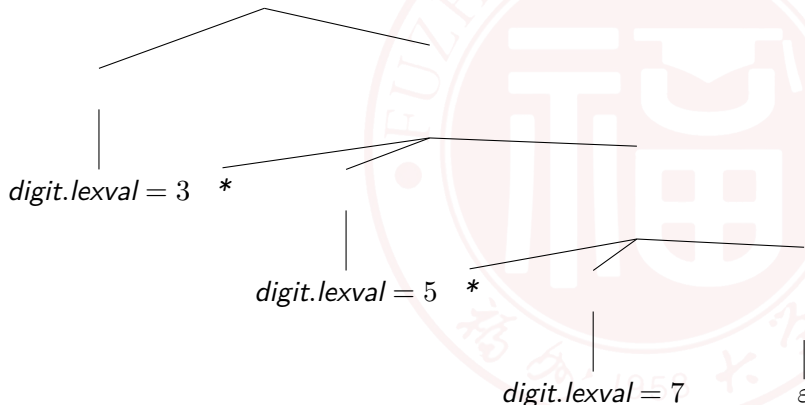
解

① 画出分析树



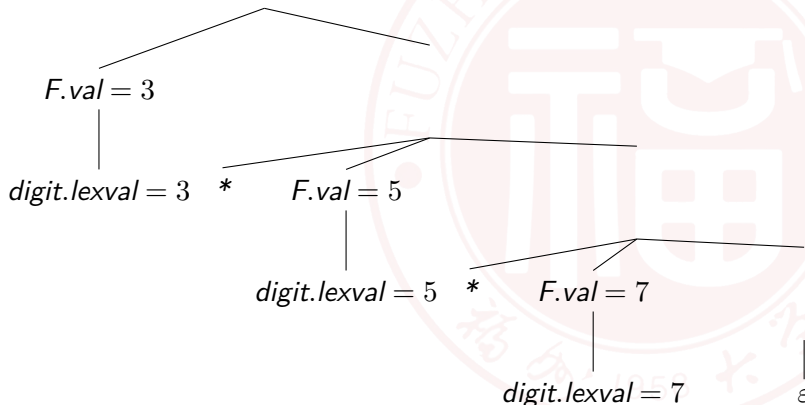
解

② 计算节点属性



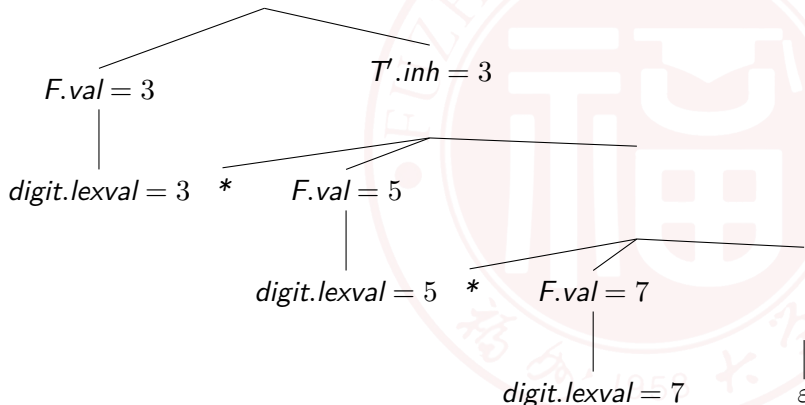
解

② 计算节点属性



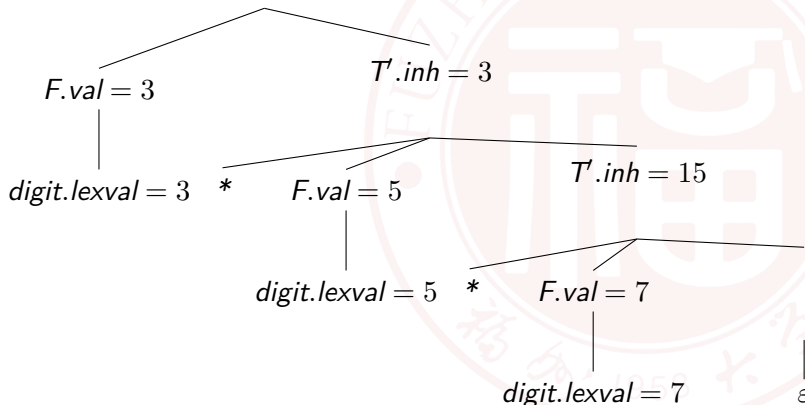
解

② 计算节点属性



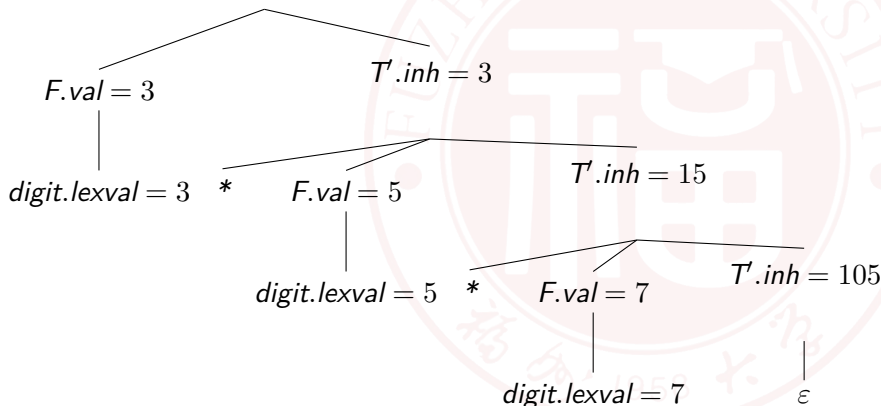
解

② 计算节点属性



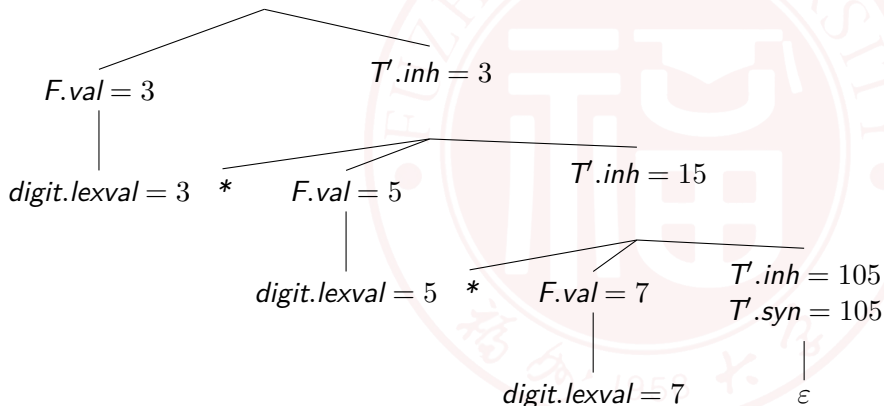
解

② 计算节点属性



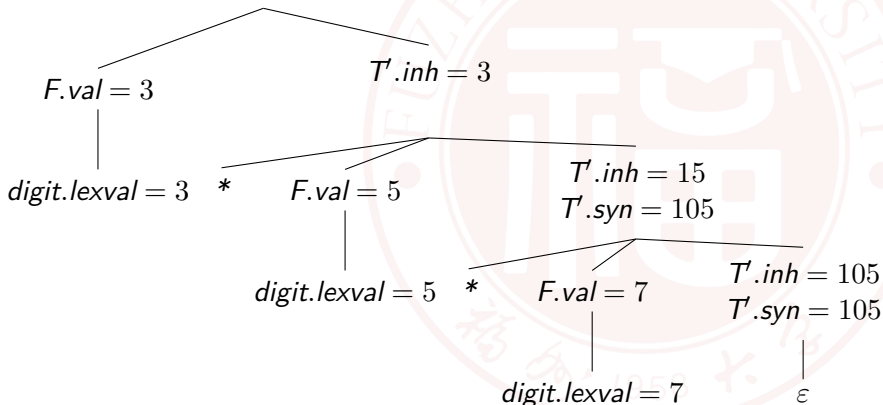
解

② 计算节点属性



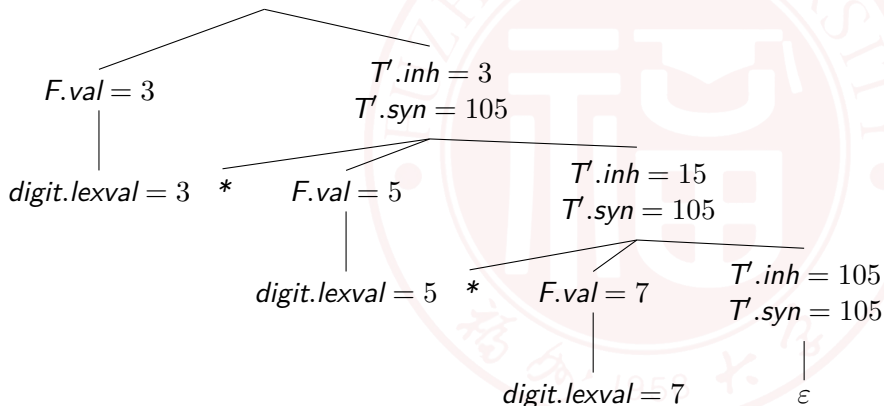
解

② 计算节点属性



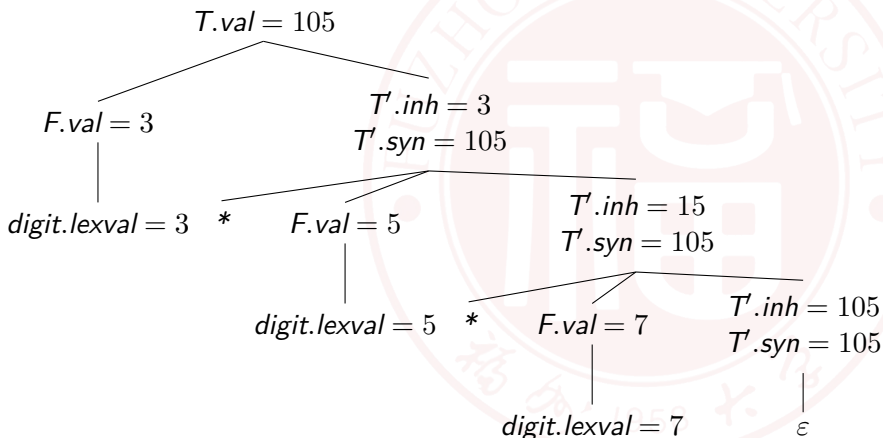
解

② 计算节点属性



解

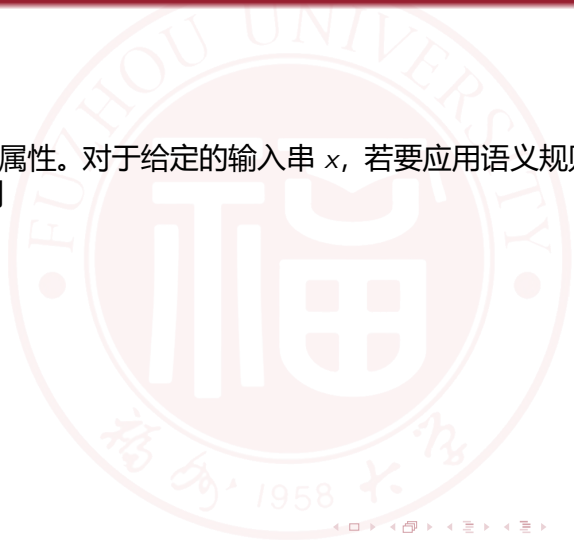
② 计算节点属性



SDD 求值顺序

SDD 为 CFG 中的文法符号设置语义属性。对于给定的输入串 x ，若要应用语义规则计算分析树中各结点对应的属性值则

- 应按照什么顺序计算属性值？



SDD 求值顺序

SDD 为 CFG 中的文法符号设置语义属性。对于给定的输入串 x ，若要应用语义规则计算分析树中各结点对应的属性值则

- 应按照什么顺序计算属性值？
 - 语义规则建立了属性之间的依赖关系，在对语法分析树节点的一个属性求值之前，必须首先求出这个属性值所依赖的所有属性值。

SDD 求值顺序

SDD 为 CFG 中的文法符号设置语义属性。对于给定的输入串 x ，若要应用语义规则计算分析树中各结点对应的属性值则

- 应按照什么顺序计算属性值？
 - 语义规则建立了属性之间的依赖关系，在对语法分析树节点的一个属性求值之前，必须首先求出这个属性值所依赖的所有属性值。
 - 文法开始符号的继承属性应事先给定。

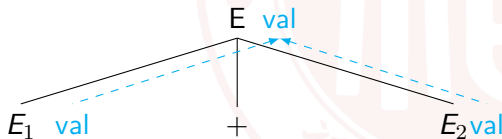
依赖图 (Dependency Graph)

- 语法树中各结点的综合属性和继承属性之间的相互依赖关系可以用**依赖图**（有向图）来描述。
- 依赖图指出语法树中各结点属性值的计算顺序。
- 特点：先有文法，后确定语义计算顺序。

依赖图

例

产生式	语义规则
$E \rightarrow E_1 + E_2$	$E.val = E_1.val + E_2.val$



依赖图构建算法

```

1  foreach 所有语法分析树的节点  $n$  do
2      |  foreach 结点  $n$  对应文法符号的每个属性  $a$  do
3          |  在依赖图中为  $a$  构建一个结点;
4      |  end
5  end
6  foreach 所有语法分析树的节点  $n$  do
7      |  foreach 结点  $n$  的产生式的每条语义规则  $b = f(c_1, c_2, \dots, c_k)$  do
8          |  |  foreach  $c_j$  do
9              |  |  从结点  $c_j$  到  $n$  构造一条有向边;
10         |  |  end
11     |  end
12 end
    
```

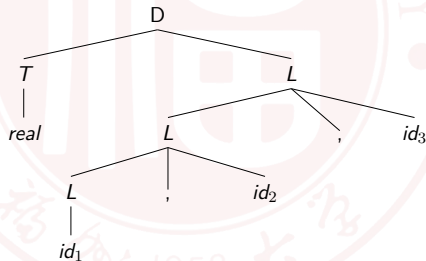
依赖图的示例

例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。



依赖图的示例

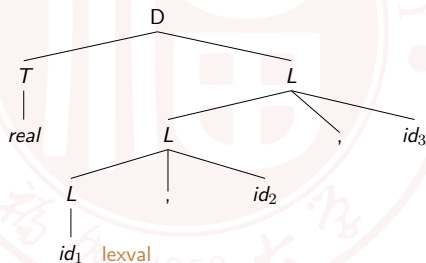
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

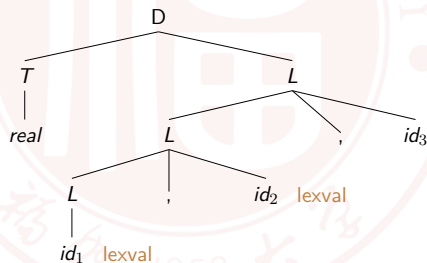
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

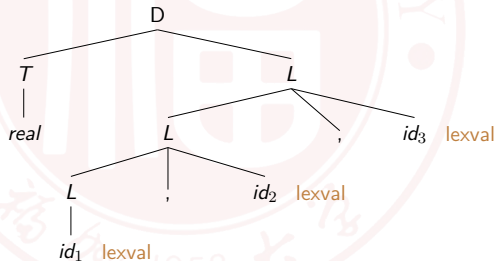
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

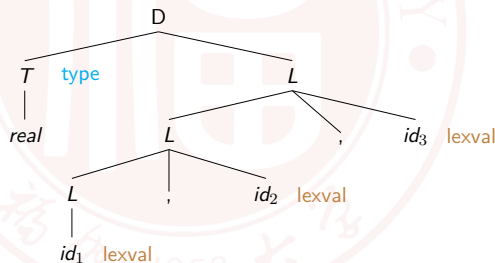
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

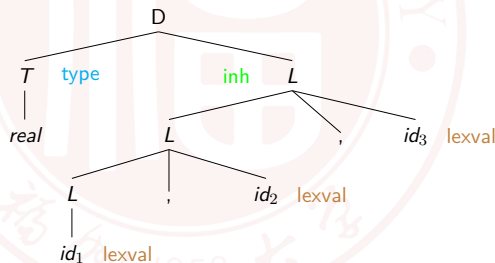
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

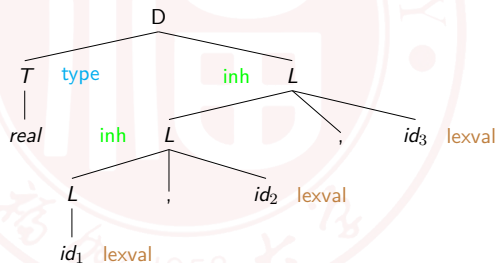
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

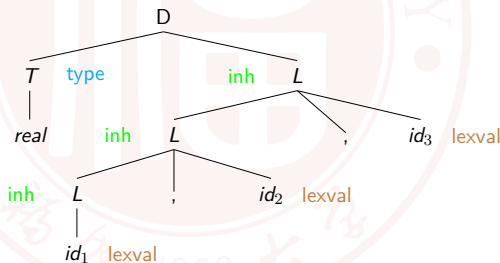
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

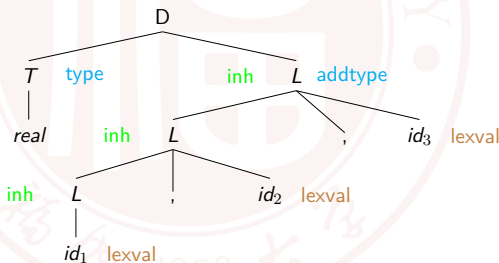
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

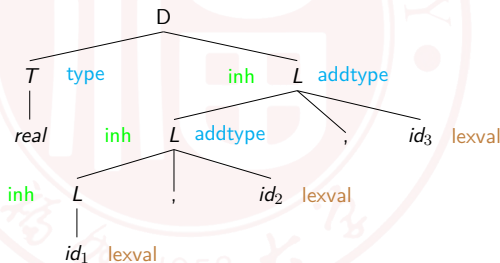
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。



依赖图的示例

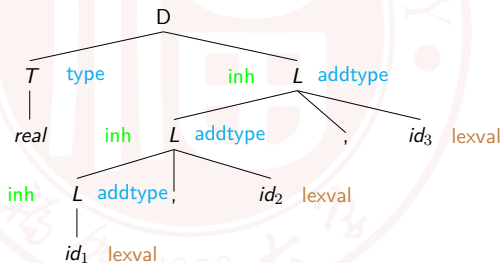
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将**继承属性 (inh)**放在分析树结点的**左边**，**综合属性 (type, lexeme)**放在分析树结点的**右边**。

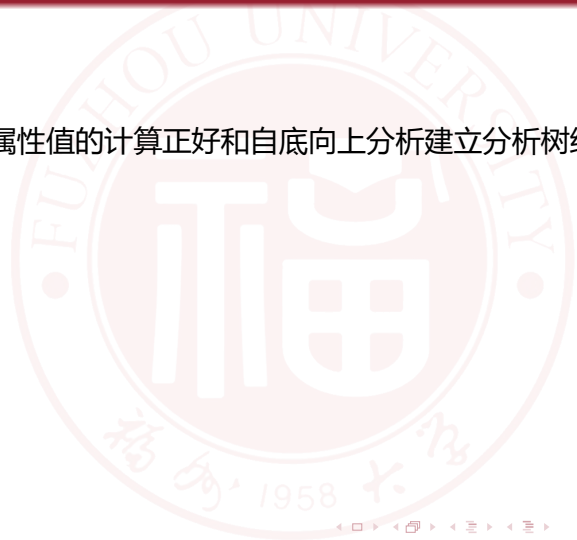


文法的良定义 (Well-defined)

- 如果一个 SDD 中不存在属性之间的**循环依赖关系**（即依赖图中不存在环），则该文法是**良定义 (Well-defined)** 的，依赖图存在拓扑排序。
- 一个依赖图的任何拓扑排序都给出一棵语法树中结点的语义规则计算的有效顺序。

SDD 求值顺序

- 对于只包含综合属性的 SDD，属性值的计算正好和自底向上分析建立分析树结点同步进行



SDD 求值顺序

- 对于只包含综合属性的 SDD，属性值的计算正好和自底向上分析建立分析树结点同步进行
- 对于同时具有继承属性和综合属性的 SDD，不能保证一定存在一个顺序来计算所有的属性值。

SDD 求值顺序

- 对于只包含综合属性的 SDD，属性值的计算正好和自底向上分析建立分析树结点同步进行
- 对于同时具有继承属性和综合属性的 SDD，不能保证一定存在一个顺序来计算所有的属性值。
- 从计算的角度看，给定一个 SDD，很难确定是否存在某棵语法分析树，使得 SDD 的属性之间存在循环依赖关系，但可以给出一组充分条件，满足这组充分条件的 SDD 是无循环依赖的。

S-属性定义

- 仅仅使用**综合属性**的 SDD 称为**S 属性的 SDD**，或**S-属性定义**、**S-SDD**。



S-属性定义

- 仅仅使用**综合属性**的 SDD 称为**S 属性的 SDD**，或**S-属性定义**、**S-SDD**。

例 (S-SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

S-属性定义

- 仅仅使用**综合属性**的 SDD 称为**S 属性的 SDD**，或**S-属性定义、S-SDD**。
- 如果一个 SDD 是 S 属性的，可以按照语法分析树节点的任何自底向上顺序来计算它的各个属性值。

例 (S-SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

S-属性定义

- 仅仅使用**综合属性**的 SDD 称为**S 属性的 SDD**，或**S-属性定义、S-SDD**。
- 如果一个 SDD 是 S 属性的，可以按照语法分析树节点的任何自底向上顺序来计算它的各个属性值。
- S-属性定义可以在自底向上的语法分析过程中实现。

例 (S-SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

L-属性定义

- L-属性定义 (也称为 L 属性的 SDD 或 L-SDD) 的直观含义: 在一个产生式所关联的各属性之间, 依赖图的边可以从左到右, 但不能从右到左 (因此取 Left 的首字母, 称为 L 属性)。
- 一个 SDD 是 L-属性定义, **当且仅当**³ 它的每个属性要么是一个**综合属性**, 要么是满足如下条件的**继承属性**: 假设存在一个产生式 $A \rightarrow X_1 X_2 \dots X_n$, 其右部符号 $X_i (1 \leq i \leq n)$ 的继承属性仅依赖于下列属性:
 - A 的**继承属性**;
 - 产生式中 X_i 左边的符号 $X_1, X_2, \dots, X_{i-1}$ 的属性;
 - X_i 本身的属性, 但 X_i 的全部属性不能在依赖图中形成环路。
- 特别地, 每个 S-属性定义都是 L-属性定义。

³if only if, iff

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例 (反例)

例

- ① 请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$A \rightarrow LM$	$L.i = l(A.i)$ $M.i = m(L.s)$ $A.s = f(M.s)$
$A \rightarrow QR$	$R.i = r(A.i)$ $Q.i = q(R.s)$ $A.s = f(Q.s)$

L-SDD 示例 (反例)

例

- 1 请指出下列语义规则中哪些是继承属性，哪些是综合属性。
- 2 请指出哪一条语义规则不是 L-SDD 的。

产生式	语义规则
$A \rightarrow LM$	$L.i = l(A.i)$ $M.i = m(L.s)$ $A.s = f(M.s)$
$A \rightarrow QR$	$R.i = r(A.i)$ $Q.i = q(R.s)$ $A.s = f(Q.s)$



① 知识点回顾

② 语法制导翻译

③ 语法制导定义

④ 语法制导翻译 (Syntax-Directed Translation, SDT)

- SDD 方案只给出属性计算的定义，没有属性计算的次序等实现细节。
- 语法制导翻译方案 (SDT) 是 SDD 的补充，它给出了语义规则计算的次序，把实现细节表示出来。

